

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 10

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: The number and the user side **3**

2 Part 2: The kernel side **6**

3 Part 3: Play with the mask **11**

Conclusion **14**

References **14**

Goal

In the previous laboratory work strace showed, from the outside, every system call that a program made on Linux. The purpose of this laboratory work is to build that tool instead of using it: a new system call, trace, is added to xv6, and it makes a process print every system call it performs. For the first time the kernel itself is changed, and not only a program that runs on top of it.

The work has three parts, and all of them follow one number, 23, from a user program down to a function inside the kernel. The first part gives the call its number and builds the user side: the number in kernel/syscall.h, the declaration in user/user.h, the stub in user/usys.pl that executes the instruction ecall, the program user/trace.c and its line in the Makefile. The second part builds the kernel side: a field in struct proc that remembers the mask of each process, the handler sys_trace in kernel/sysproc.c, two lines in kernel/syscall.c that put the function at place 23 of the table, a table of names and the print itself, and one line in fork so that a child inherits the mask of its parent. The third part plays with the mask, which is a number read in binary, in which the bit N switched on means print the call number N.

The work was done in the tree ~/xv6-riscv, built and started with make qemu and left with Ctrl-A X. One warning of the task sheet had to be respected: this tree is newer than the pages of MIT 6.1810 found on the internet, where SYS_trace is 22 and the kernel prints with printf. Here the number 22 is already taken by sync, so the new call received the number 23, and the print function of the kernel is called printk and not printf.

Part 1: The number and the user side

A system call does not exist until both sides agree on a number: the user program must be able to ask for it, and the kernel must be able to recognize it. This part performs the five edits of the user side, after which the number travels correctly but nothing answers it yet.

```
#define SYS_open   15
#define SYS_write  16
#define SYS_mknod  17
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_sync   22
#define SYS_trace  23
```

Figure 1.1: The number of the new call in kernel/syscall.h

kernel/syscall.h – one line was added at the end of the list of numbers: #define SYS_trace 23. The file is included both by the kernel and, indirectly, by the generated stub, so the same constant names the same number on both sides of the door. The

numbers before it are those of the existing calls, from SYS_open 15 and SYS_write 16 to SYS_close 21 and SYS_sync 22, and they are also the numbers used later by the mask.

```
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
int trace(int);
```

Figure 1.2: The declaration of trace in user/user.h

user/user.h – the line int trace(int); was added after int sync(void);. This is only a declaration: it tells the compiler that a function with this name and this signature exists, so that a user program may call it, and it says nothing about where the function is. The body is not written by hand at all, because it is generated during the build.

```
entry("mkdir");
entry("chdir");
entry("dup");
entry("getpid");
entry("sbrk");
entry("pause");
entry("uptime");
entry("sync");
entry("trace");
```

Figure 1.3: The entry added to user/usys.pl

user/usys.pl – the line entry("trace"); was added at the end. This Perl script is run at build time and writes the file user/usys.S, in which every entry becomes three instructions in assembly: li a7, SYS_trace, which places the number 23 in the register a7, then ecall, which is the instruction that leaves the user mode and enters the kernel, and then ret. The register a7 is the only place where the kernel looks to know which call is being asked for, and the arguments travel in the registers a0 to a5.

```
 GNU nano 8.7.1                    user/trace.c *
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
if (argc < 3) {
fprintf(2, "usage: trace mask command [args...]\n");
exit(1);
}
```

Figure 1.4: The beginning of user/trace.c in the editor

user/trace.c – the program checks that it received at least two arguments, the mask and the command, and otherwise prints its usage on the descriptor 2 and ends with the status 1. It then calls trace with the mask converted from text to a number by atoi, which switches the tracing on for itself, and finally calls exec with the rest of the arguments, so that the process becomes the requested command. The mask survives this transformation because exec does not create a new process: it only replaces the program inside the one that already exists, together with its memory image, while the structure that the kernel keeps for the process, with its pid and now its mask, stays the same. The source is given below.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

```
$U/_mkdir\
$U/_rm\
$U/_sh\
$U/_stressfs\
$U/_usertests\
$U/_grind\
$U/_wc\
$U/_zombie\
$U/_trace\
$U/_logstress\
$U/_forphan\
```

Figure 1.5: The line $U/_trace\ added to UPROGS in the Makefile

Makefile – UPROGS – the line $U/_trace\ was added after $U/_zombie\, with a TAB at the beginning and with nothing after the backslash, not even a space. Without it the program would be compiled by nothing and would never be copied into the image of the file system, so the shell of xv6 would answer exec trace failed.



Figure 1.6: Check 1: the number 23 reaches the kernel, which has no function for it yet

Check 1 – $ trace 32 grep hello README – the answer of the kernel is 3 trace: unknown sys call 23, followed by trace: failed, which is exactly the expected result at this stage and is good news rather than an error. It proves that the whole user side works: the program called the stub, the stub put 23 into a7 and executed ecall, and the kernel received the number. The kernel then looked for the place 23 in its table of functions, found nothing there, printed the message with the pid 3 in front and returned −1, which trace.c reported as a failure. The five edits are summarized in Table 1.

File	What was added	Why it is needed
kernel/syscall.h	#define SYS_trace 23	gives the call a number known to both sides
user/user.h	int trace(int);	lets a user program call it by name
user/usys.pl	entry("trace");	generates the stub: li a7, 23; ecall; ret
user/trace.c	the whole program	sets the mask, then becomes the command with exec
Makefile	$U/_trace\ in UPROGS	compiles the program and puts it on the xv6 disk

Table 1: The five edits of the user side

Table 1 shows that none of these files contains any logic of the new call: they only carry the number from the name written by the programmer down to the register a7, which is the only thing the kernel reads.

Part 2: The kernel side

The number now arrives at the kernel and is refused. This part writes what happens when it arrives: where the mask is stored, which function receives the call, how the table learns about it, and where the line of the trace is printed.

```
  struct inode *cwd;          // Current directory
  char name[16];              // Process name (debugging)
  int trace_mask;
};
```

Figure 2.1: The new field of struct proc in kernel/proc.h

kernel/proc.h – the field int trace_mask; was added to struct proc, after the field name. The mask belongs to each process separately and not to the kernel as a whole, which is

what makes it possible to trace one command while the shell and all the other processes stay silent. The structure is the place where the kernel keeps everything it knows about a process, such as its pid, its page table, its open files and its current directory, so the mask simply becomes one more of these properties.



Figure 2.2: The handler sys_trace in kernel/sysproc.c

kernel/sysproc.c – the function sys_trace is the kernel side of the call. It does not receive its argument as an ordinary parameter, because the argument was left by the user program in the register a0 and crossing into the kernel changes the stack; it must therefore be fetched with the helper argint(0, &mask), where 0 means the first argument. The value is then stored on the process returned by myproc(), which is the process that made the call, and the function returns 0, the value that the user program receives from trace. This is also why trace.c can test whether the call failed.

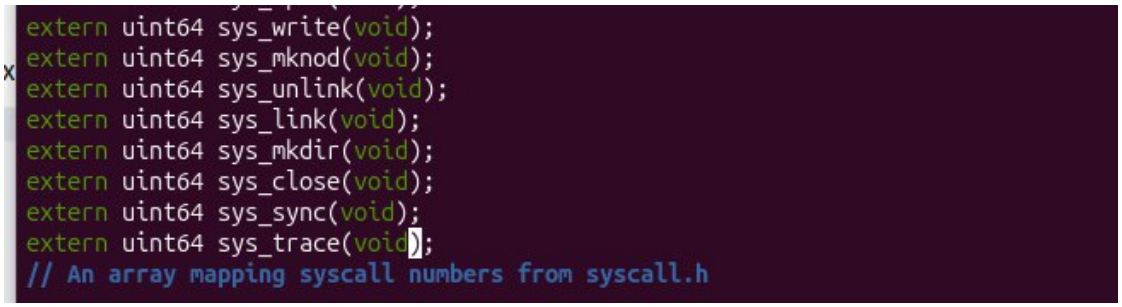


Figure 2.3: The declaration of sys_trace in kernel/syscall.c

kernel/syscall.c – extern – the line extern uint64 sys_trace(void); was added after the one of sys_sync. The handler lives in another file, so this line tells the compiler that such a function exists somewhere and that the linker will find it; without it the file would not compile, because the name used in the table below would be unknown.

```
  [SYS_mknod]   = sys_mknod,
  [SYS_unlink]  = sys_unlink,
  [SYS_link]    = sys_link,
  [SYS_mkdir]   = sys_mkdir,
  [SYS_close]   = sys_close,
  [SYS_sync]    = sys_sync,
  [SYS_trace]   = sys_trace,
  // clang-format on
};
```

Figure 2.4: The entry of the new call in the table of the kernel

kernel/syscall.c – the table – the line [SYS_trace] = sys_trace, was added to the array syscalls, which maps every number to the function that handles it. This single line is what makes the message of Check 1 disappear, because the place 23 of the array is no longer empty. The notation with the number inside brackets places the element at that exact index, so the order of the lines in the source does not matter.

```
  [SYS_trace]   = sys_trace,
  // clang-format on
};

static char *syscallnames[] = {
[SYS_fork] "fork", [SYS_exit] "exit", [SYS_wait] "wait",
[SYS_pipe] "pipe", [SYS_read] "read", [SYS_kill] "kill",
[SYS_exec] "exec", [SYS_fstat] "fstat", [SYS_chdir] "chdir",
[SYS_dup] "dup", [SYS_getpid] "getpid", [SYS_sbrk] "sbrk",
[SYS_pause] "pause", [SYS_uptime] "uptime", [SYS_open] "open",
[SYS_write] "write", [SYS_mknod] "mknod", [SYS_unlink] "unlink",
[SYS_link] "link", [SYS_mkdir] "mkdir", [SYS_close] "close",
[SYS_sync] "sync", [SYS_trace] "trace",
};
```

Figure 2.5: The table of names added above the function syscall()

kernel/syscall.c – the names – the kernel knows only numbers, so a second table, syscallnames, was added just above the function syscall(), giving the text of every name from fork and exit to sync and trace. The position above the function matters: in C a name must be declared before it is used, and the print inside syscall() reads this table. Its index is the same number as in the table of the functions, which is why the two stay aligned automatically.

```
int num;
struct proc *p = myproc();

num = p->trapframe->a7;
if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
  // Use num to lookup the system call function for num, call it,
  // and store its return value in p->trapframe->a0
  p->trapframe->a0 = syscalls[num]();
  if ((p->trace_mask >> num) & 1) // new
      printk("%d: syscall %s -> %ld\n",
      p->pid, syscallnames[num], p->trapframe->a0);
} else {
  printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
  p->trapframe->a0 = -1;
}
```

Figure 2.6: The two new lines inside syscall(), after the call is performed

kernel/syscall.c – the print – the two lines were placed immediately after p->trapframe->a0 = syscalls[num]();, which is the line that actually performs the call, and not before it. The order is essential, because the value printed is the result of the call, which exists in a0 only after it has returned. The condition ((p->trace_mask » num) & 1) shifts the mask to the right by the number of the call and keeps the last bit, so it is true only when the bit that corresponds to that call is switched on. The print itself uses printk and not printf, which is the name of the print function of the kernel in this tree, and it writes the pid, the name taken from the table and the returned value.

```
GNU nano 8.7.1                    kernel/proc.c *
  if (p->ofile[i])
    np->ofile[i] = filedup(p->ofile[i]);
np->cwd = idup(p->cwd);

safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
pid = np->pid;
```

Figure 2.7: The inheritance of the mask in fork(), in kernel/proc.c

kernel/proc.c – fork – the line np->trace_mask = p->trace_mask; was added right after the one that copies the name of the process. A child created by fork receives a fresh entry in the process table, so without this line its mask would be zero and the child would print nothing. With it the mask is inherited exactly like the name, the open files or the current directory, and a traced command keeps being traced through all the processes it creates.



Figure 2.8: Check 2: trace 32 grep hello README prints the four read calls

Check 2 – $ trace 32 grep hello README – after Ctrl-A X and a new make qemu the call works. The trace prints four lines, all of them with the pid 3: syscall read gives 1023, then 965, then 453 and finally 0. The three first numbers are the pieces in which grep read the file, and their sum, $1023+965+453 = 2441$, is exactly the size of README, which was also the size measured in an earlier laboratory work; the last 0 is not a piece of zero bytes but the way the kernel reports the end of the file, which is how grep learns that it must stop reading. The seven edits of the kernel side are summarized in Table 2.

File	**What was added**	**What it does**
kernel/proc.h	int trace_mask; in struct proc	every process remembers its own mask
kernel/sysproc.c	the function sys_trace	reads the argument with argint and stores it
kernel/syscall.c	extern uint64 sys_trace(void);	announces the handler written in another file
kernel/syscall.c	[SYS_trace] = sys_trace,	fills the place 23 of the table of functions
kernel/syscall.c	the table syscallnames	turns the number back into a name for the print
kernel/syscall.c	the condition and the printk	prints the line, after the call has returned
kernel/proc.c	np->trace_mask = p->trace_mask;	the children inherit the mask of the parent

Table 2: The edits of the kernel side and their role

Table 2 shows that the new call needed only one real function, while everything else is the plumbing that connects a number to that function and turns the number back into a readable name.

Part 3: Play with the mask

Questions:

- In trace 32 grep hello README why do you see only read, and not open or close?
- In forkforkfork, three different pids print lines. Which line of code from Part 2 makes that possible?
- In trace 65536 echo hi the trace line is printed between "hi" and the new line. Why? How many times does echo call write?
- Which mask traces both open (15) and close (21), and nothing else?

Answers:

- Because the mask is read bit by bit and 32 has only one bit switched on. In binary 32 is 2^5, that is 100000, so only the bit number 5 is on, and 5 is the number of SYS_read. The condition of the print, which shifts the mask to the right by the number of the call and keeps the last bit, is therefore true only for read; for open, whose number is 15, and for close, whose number is 21, the corresponding bits are zero and the kernel performs the calls normally but prints nothing. The command really does open and close the file, as the next experiment proves, and the silence of the trace is a choice of the mask and not an absence of the calls.
- The line np->trace_mask = p->trace_mask; added in fork, in kernel/proc.c. The mask is stored in struct proc, which belongs to one process only, so a child born from fork would start with a mask of zero and would print nothing. Because the parent copies its mask into the child at the moment of the fork, every new process of the chain keeps tracing, and the pids that appear in the trace are those of the successive children.
- Because echo calls write twice and the trace line is printed after the call has returned, not before. The first write hands the two characters of the word to the kernel, so the screen already shows hi, and only then the kernel prints its line, which therefore begins on the same line, right after the word: hi67: syscall write -> 2. The second write sends the single character of the new line, which ends that line, and is followed by the second trace line, 67: syscall write -> 1. The returned values, 2 and 1, are the numbers of bytes accepted by the kernel and confirm the count: two calls, one for the argument and one for the newline.
- The mask 2129920. The bit of open is the number 15 and the bit of close is the number 21, so the mask is $2^{15} + 2^{21} = 32768 + 2097152 = 2129920$, and no other bit is switched on. The masks used in this part are collected in Table 3.



Figure 3.1: All the bits on: the whole life of the command is printed

$ trace 2147483647 grep hello README – the number 2147483647 is the largest positive value of a signed 32 bit integer, $2^{31} - 1$, so all its bits are on and every call is printed. The trace now shows the whole life of the command, with the pid 4: first syscall trace returns 0, which is the call made by trace.c on itself, then syscall exec returns 3, the moment at which the process stops being trace and becomes grep, then syscall open returns 3, which is the descriptor given to README, then the four read lines already seen, and finally syscall close returns 0. The two calls that the mask 32 had hidden, open and close, are indeed there, which answers the first question from the other side. It is also worth noting that the line of trace itself appears: the mask was already stored when that call returned, so the call could print its own line.



Figure 3.2: Only fork, traced through a chain of children

$ trace 2 usertests forkforkfork – the mask 2 is 2^1, and 1 is the number of SYS_fork, so only the creation of processes is printed. The test creates children until the process table of the kernel is full, and the trace shows the chain growing: the first line comes from the pid 5, then the pid 6 prints that its fork returned 7, the pid 7 prints the values 8, 9, 10, 11 and 12, and the pid 8 prints 13. Several different pids appear because every child inherited the mask, and each printed line reports a fork made by that particular process; the value after the arrow is the pid of the newly created child, which the parent receives from fork. When the table is finally full, fork returns -1 and the test ends normally.



Figure 3.3: Only write: the trace line lands between the word and the new line

$ trace 65536 echo hi – the mask 65536 is 2^{16}, and 16 is the number of SYS_write, so only the writes are printed. The result is the small typographic accident described in the third question: the output begins with hi, immediately followed on the same line by 67: syscall write -> 2, and the second trace line, 67: syscall write -> 1, comes after the new line has been written. The two lines together show that echo performs exactly two system calls to print two characters and a new line, and that the trace of the kernel and the output of the program share the same screen, which is why they interleave.

Mask	**Bits on**	**Calls traced**	**What was observed**
32	5	read	the four reads of grep: 1023, 965, 453, 0
2147483647	0 to 30	all of them	trace, exec, open, four reads, close
2	1	fork	the chain of children of forkforkfork
65536	16	write	the two writes of echo: 2 and 1 bytes
2129920	15 and 21	open and close	the answer to the fourth question

Table 3: The masks used and the calls each of them selects

Table 3 shows that one integer is enough to choose any combination of the calls, because each call owns one bit of it, and that the mask does not change what the program does but only what the kernel reports.

Conclusion

The laboratory work added a new system call to xv6 and, with it, built from the inside the tool that the previous work had only used from the outside. The first part followed the number 23 through the user side: the constant SYS_trace in kernel/syscall.h, the declaration in user/user.h, the entry in user/usys.pl that generates the three instructions li a7, 23, ecall and ret, the program user/trace.c that sets the mask on itself and then becomes the command with exec, and the line in UPROGS that puts the program on the disk of xv6. The check at the end of this part printed trace: unknown sys call 23, which proved that the number had travelled correctly and that only the answer was missing.

The second part wrote that answer. The field trace_mask was added to struct proc, so that the mask belongs to one process; the handler sys_trace read its argument with argint, because the argument arrives in the register a0 and not on the stack, and stored it on the process returned by myproc(); the table of functions received the entry [SYS_trace] = sys_trace, which is the line that makes the error of the first check disappear; a table of names was placed above syscall(), since the kernel knows only numbers; and two lines inside syscall(), placed after the call so that the returned value already exists in a0, print the pid, the name and the result with printk whenever the bit of the call is on in the mask. One more line, in fork, copies the mask into the child. The check then printed the four reads of grep, 1023, 965, 453 and 0, whose sum is the size of README, 2441 bytes.

The third part showed that the mask is simply an integer read in binary. The mask 32 selected only read, the mask 2147483647 showed the whole life of the command, from its own trace call through exec, open, the reads and close, the mask 2 followed the chain of children of forkforkfork and proved that the line added in fork is what lets several pids print, and the mask 65536 showed the two writes of echo and why the trace line lands between the word and the new line. The mask that would select only open and close is 2129920. Overall, the work gave a practical understanding of how a system call is built, of the role of the register a7 and of the instruction ecall, and of how little code separates a number from a working feature of a kernel; all the objectives of the laboratory work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *Xv6, a simple Unix-like teaching operating system: the book and the source code (MIT 6.1810)*
   `https://pdos.csail.mit.edu/6.828/xv6`
3. *Lab: system calls (MIT 6.1810), the trace exercise*
   `https://pdos.csail.mit.edu/6.828/2023/labs/syscall.html`
4. *strace(1) – trace system calls and signals, the Linux tool built here for xv6*
   `https://man7.org/linux/man-pages/man1/strace.1.html`